

PYCON US 2026 - 30 MIN

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX
- told from the workflow of a statistician.

Wenxin Jiang and Jian Yin - Department of Biostatistics, City University of Hong Kong

Trust → speed → scale

Prove the statistic, accelerate the bottleneck, then test scale.

Simulation first

Controlled truth before runtime charts.

Python interface

NumPy, Numba, Python 3.14, and JAX/A100 solve different bottlenecks.

For statisticians, speed is useful only after trust

Optimization enters after reference behavior is defined, stressed, and checked.

Statistical specification evolves

Simulation exposes assumptions, hard cases, and stable outputs.

Reference code is scientific specification

Readable Python or NumPy defines the validation target.

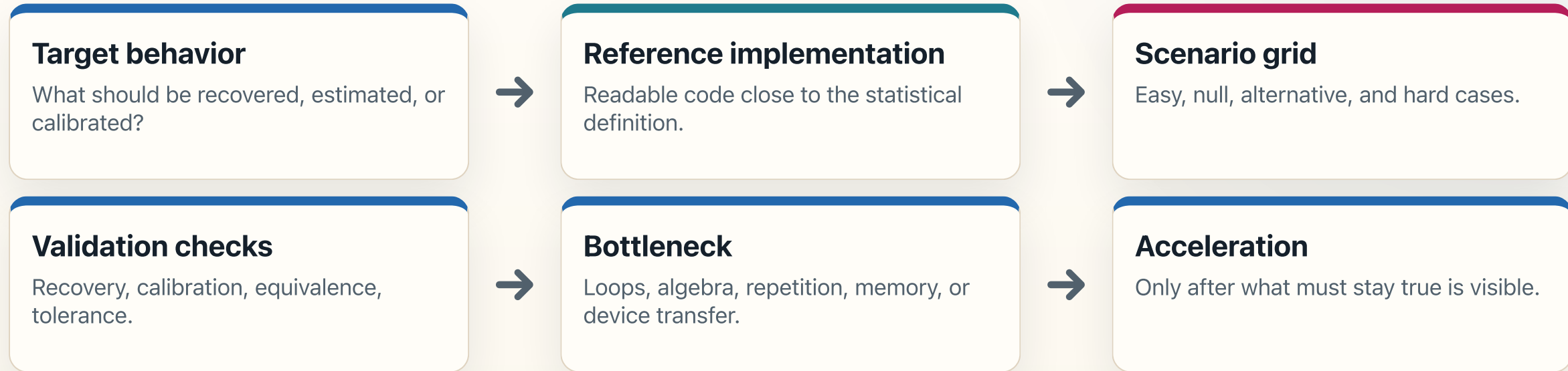
Acceleration is conditional

Faster paths must answer the same statistical question on controlled scenarios.

A speedup only counts if the optimized code answers the same statistical question.

Simulation creates an answer key

In real biomedical data, ground truth is often unavailable.



Simulation-driven statistical computing is statistical CI

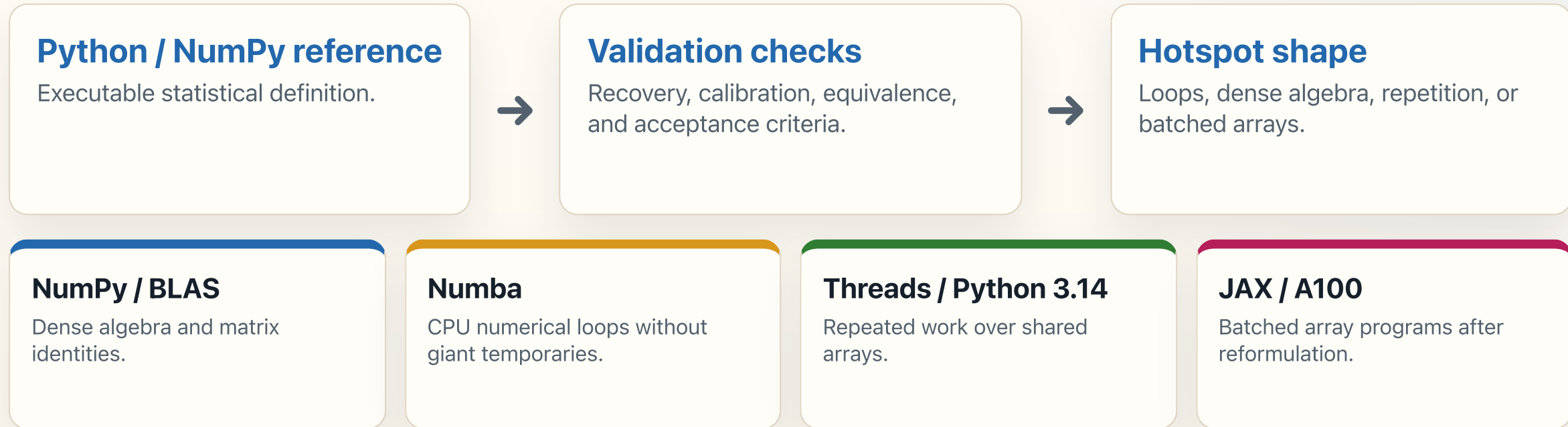
The testing instinct is familiar; the oracle is statistical behavior rather than one expected scalar.

How simulation maps to software testing

Unit tests	Fixed-seed reference equivalence.
Property tests	Null invariants and calibration.
Stress tests	Imbalance, outliers, high dimension.
Load tests	Scale N, d, K, p, R, workers, and batch sizes.
Regression tests	Acceptance criteria keep passing.

Python is the control plane; the hotspot can move to the right engine

Keep the model readable, then move the measured hotspot into the smallest engine that preserves the validation target.



Two workloads, two statistical computing shapes

k-means = iterative model fitting

assign → update centroids → repeat

Each step depends on the previous centroids.

distance loops

temporaries

compiled kernels

permutation test = resampling inference

shuffle labels → compute statistic → repeat

Independent draws become expensive across many features.

shared arrays

batching

W @ X

Evidence ladder: trust, scale, acceleration

MacBook Air

Trust tier: controlled simulation and reference equivalence.



Linux server CPU

Scale tier: larger shapes, parallelism, and memory accounting.



A100 GPU

Accelerator tier: only when the pipeline is batched and device-resident.

Correctness before runtime

Every timing row carries a validation status.

Warm timing plus memory

Cold calls, warm medians, and RSS are logged separately.

Unavailable is data

JIT, GPU sync, and interpreter state are explicitly labeled.

WORKLOAD 1

k-means as iterative model-fitting pressure

Same data and initialization; vary geometry and scale.

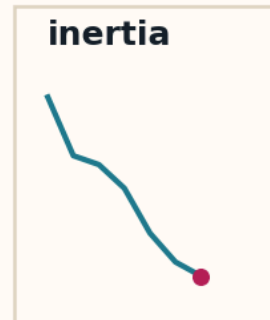
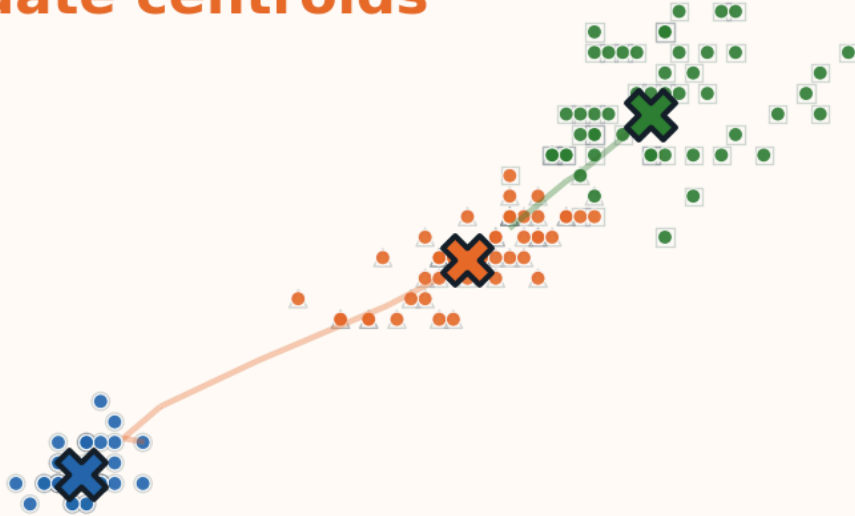
How k-means moves

Assignment and centroid update form an iterative dependency loop.

Iris petal features - Lloyd iteration 7

assign -> update centroids -> repeat

update centroids



Species outlines are subtle;
true labels are not used.

One Lloyd loop

1. assign each point to the nearest centroid
2. update centroids from assigned points
3. repeat until stable

Each iteration depends on the previous centroids.

This is why explicit loops, temporary arrays, and distance computation connect naturally to Numba, NumPy matmul, and JAX/A100.

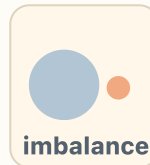
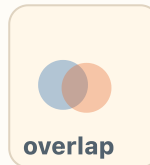
A simple algorithm exposes common iterative pressure

Validation target

fixed initialization, inertia, ARI recovery

Challenging scenarios

overlap, imbalance, outliers, high dimension



Compute pressure

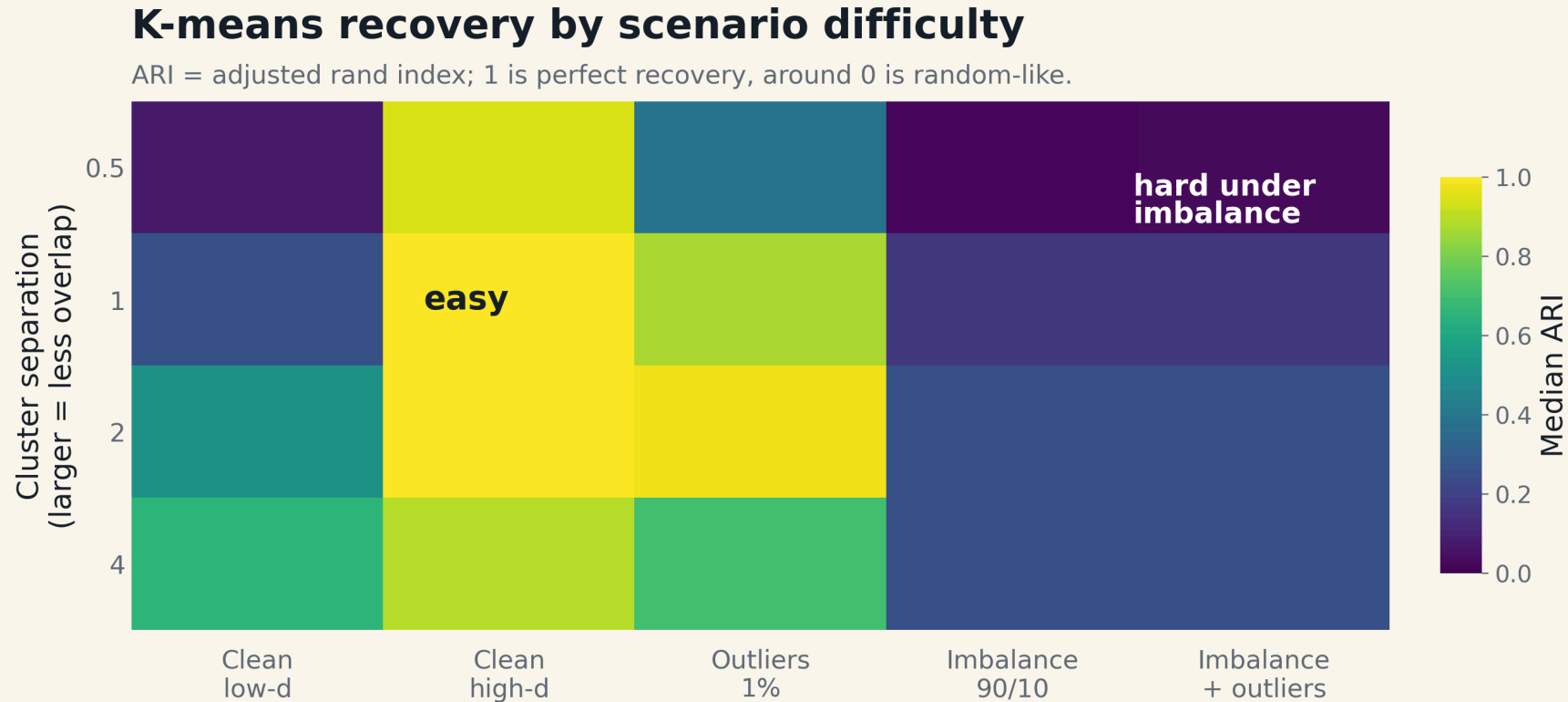
distance loops, $O(N*K*d)$ temporaries, repeated updates

Numba loops

NumPy matmul distance algebra

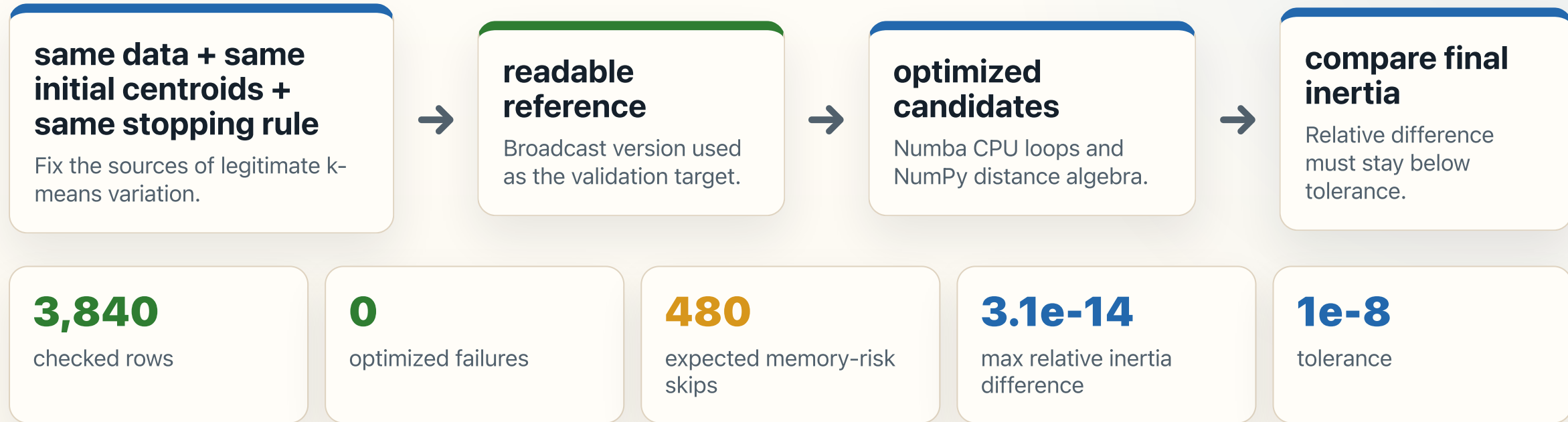
JAX/A100 large regular loops

First ask: did k-means recover the simulated clusters?



Do not celebrate speed on scenarios where recovery is poor.

Before timing: does optimized k-means reproduce the reference?



Memory-risk skips are expected skips from unsafe reference-broadcast rows, not optimized failures.

Server k-means: acceleration is shape-dependent

Numba: explicit CPU loops

Good when loop overhead and temporary avoidance dominate.

BLAS: distance algebra

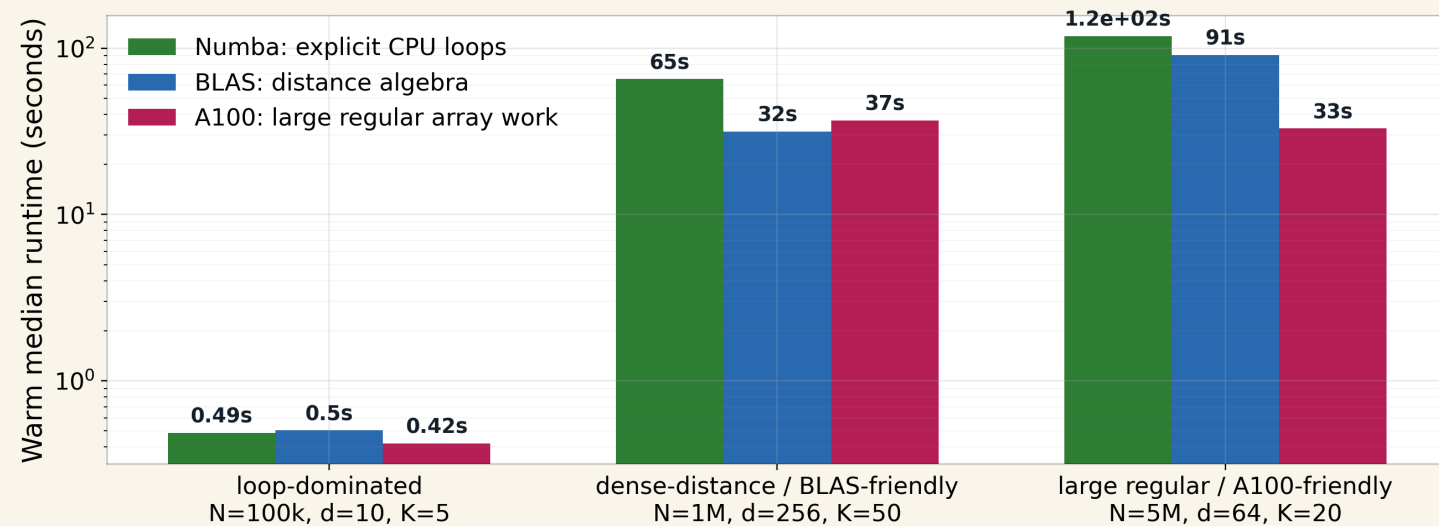
Good when the rewrite maps to dense matrix work.

A100: large regular array work

Helpful only in measured accelerator-shaped rows.

Representative k-means shapes

Validated server rows; separation=2.0 for CPU rows, fixed seeds, same initialization policy within each row.



A100 is not universally faster; shape and implementation decide.

Validated server rows only; same shape, seed set, and initialization policy within each comparison.

The tool choice follows the simulation signal

Unstable recovery

Do not optimize yet; the validation target is not ready.

Scalar loops

Use Numba to compile explicit CPU kernels.

Dense distance algebra

Use NumPy/BLAS when the rewrite removes large temporaries.

Huge regular work

Use JAX/A100 only after enough batched work exists.

k-means stands in for a large class of iterative statistical algorithms: EM, coordinate descent, optimization loops, and simulation-based estimators.

WORKLOAD 2

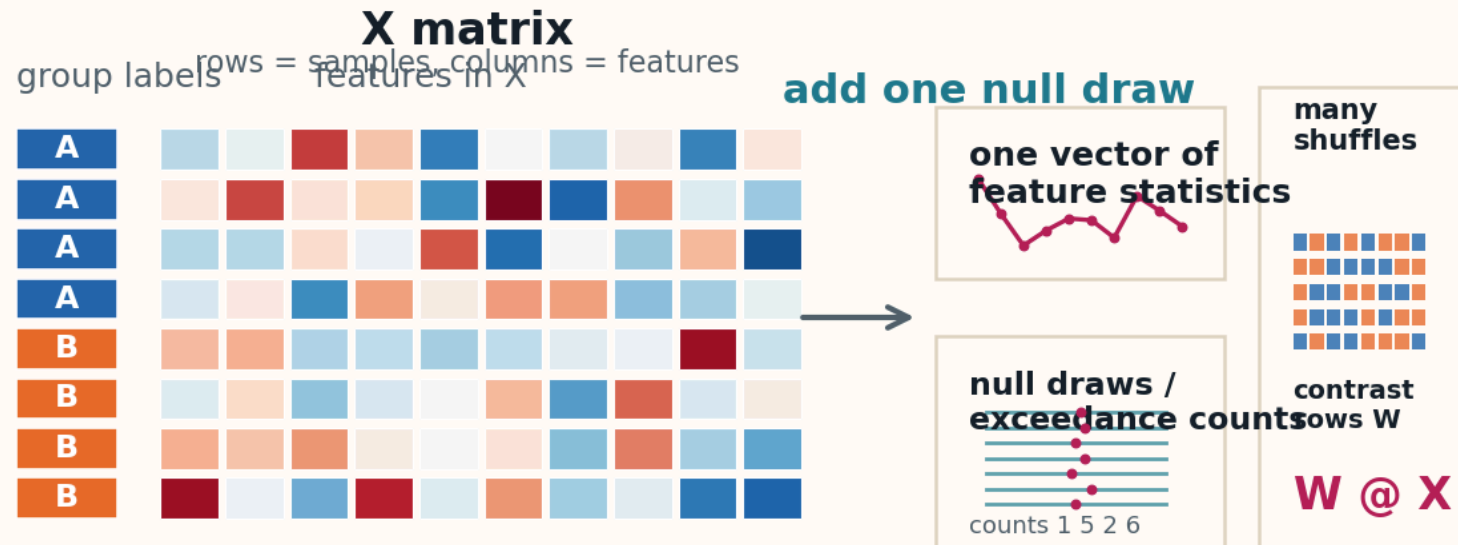
Permutation tests as resampling inference pressure

A simple inference procedure becomes expensive when we need thousands of null draws across a high-dimensional features matrix.

How a permutation test scales

One simple statistic becomes many independent null draws.

Permutation test: one shuffle, many feature statistics



One resampling loop

1. shuffle group labels
2. recompute the statistic
3. repeat many times to form the null distribution

The repetitions are independent; the pipeline is not free.

This is why permutation tests connect to threads/processes, Numba kernels, and JAX/A100 matrix formulations.

WHY PERMUTATION TESTS WORK HERE

High-dimensional testing: one label shuffle, many features

Rows are samples. Columns are features.

sample 1 **A**

sample 2 **A**

sample 3 **A**

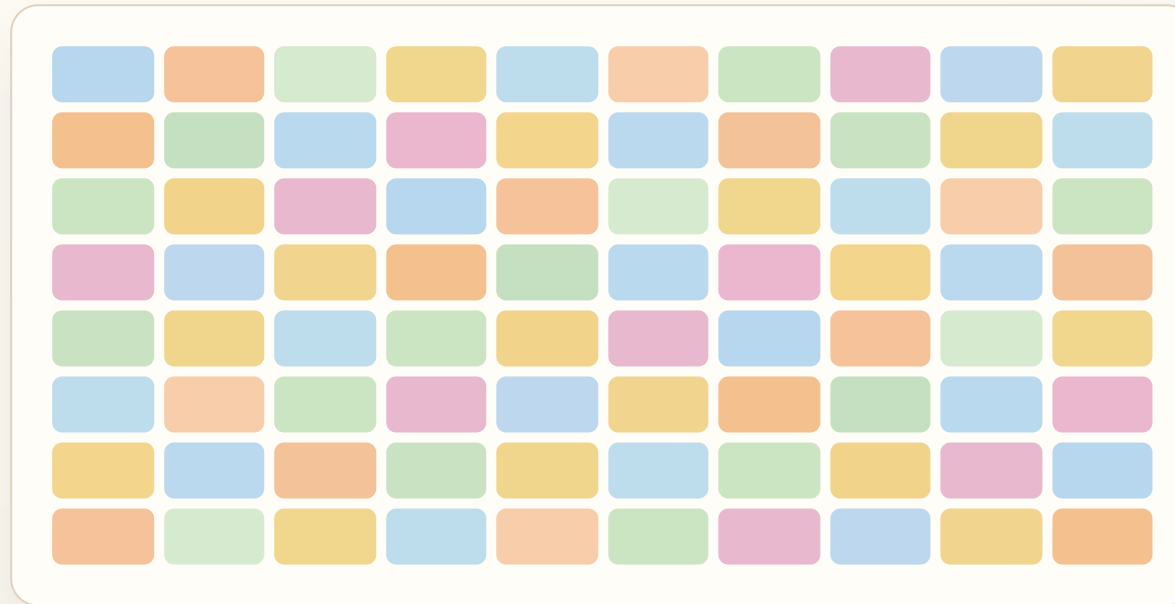
sample 4 **A**

sample 5 **B**

sample 6 **B**

sample 7 **B**

sample 8 **B**



features

genes, biomarkers,
voxels

Statistical algebra changes the kernel, not the statistic

Reference view

- 1 shuffle labels
- 2 compute group difference
- 3 repeat R times

same
permutation
stream

Batched view

- 1 encode shuffles as contrast rows W
- 2 compute $W @ X$
- 3 stream exceedance counts

Only valid after equivalence checks.

Equivalence check: loop reference vs matrix formulation

Same simulated data, same permutation stream, same p-value definition.

Context

18 workloads × 5 seeds

Reference

readable loop formulation

Candidate

matrix formulation

0.0

max |p diff|

9.4e-16

max |stat diff|

45

expected memory-risk
skips

0

failed rows

450

admitted rows

Admitted to runtime comparison.

Null calibration gate: type-I error stays near alpha

Under the null, the type-I error should stay close to nominal $\alpha = 0.05$.

Why calibration matters

Equivalence checks implementation. Calibration checks statistical behavior.

Observed check

Observed type-I error = 0.051 at nominal alpha 0.050.

What this unlocks

Now runtime comparisons become meaningful.

calibrated **0.051**
observed type-I error estimate

0.050

nominal alpha

100

null replicates

500

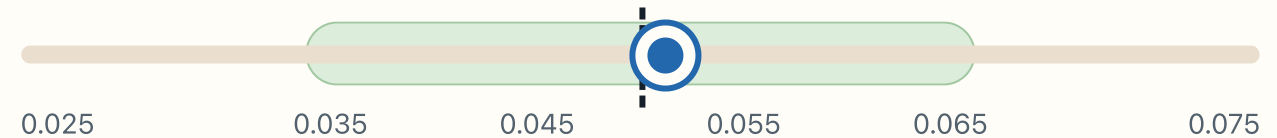
n

1,000

p and R

Calibration ruler

expected Monte Carlo band 0.036-0.064



Dashed: nominal 0.05

Dot: observed mean 0.051

Expected band: $0.05 \pm 1.96 * \sqrt{0.05 * 0.95 / 1000}$.

Fast code is useful only if null calibration remains correct.

LOCAL VALIDATION SCALE

Local validation scale: methods are close enough

Scope

MacBook CPU validation tier; not a server/GPU leaderboard.

Same permutations

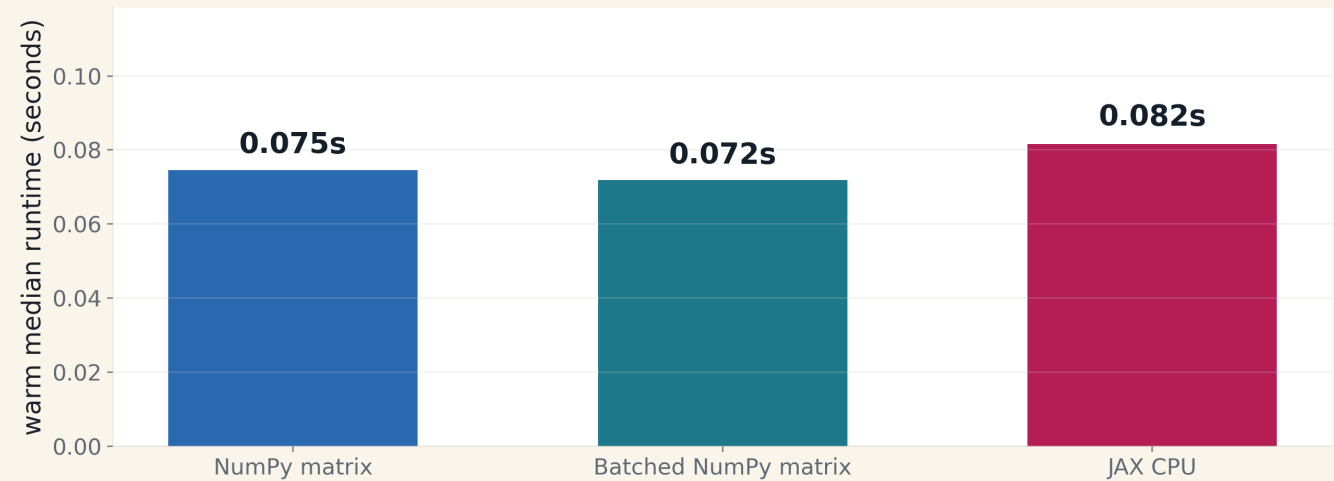
Compared only after p-value equivalence checks.

Practical rule

At validation scale, readability and correctness should dominate.

Local validation scale: methods are close enough

MacBook CPU validation tier; n=500, p=1,000, R=1,000; same validated p-values.



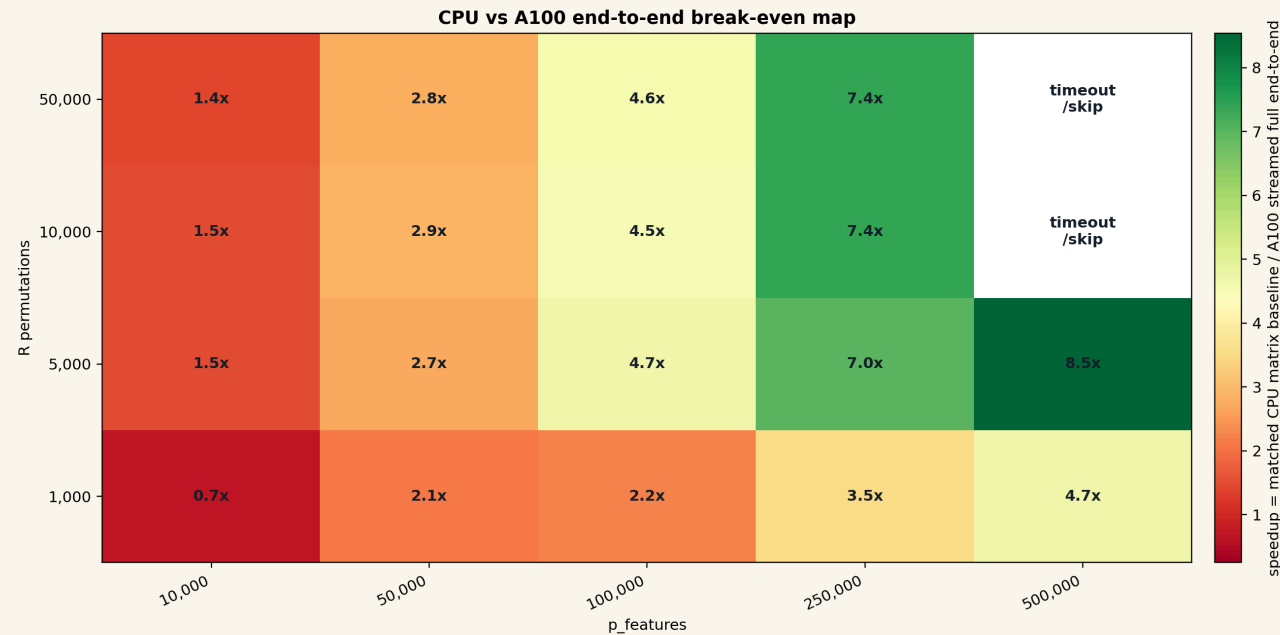
Choose the clearest correct implementation until the bottleneck is real.

At validation scale, choose the clearest correct implementation; optimize after the bottleneck is real.

DECISION MAP

When does GPU help for permutation tests?

A100 wins only after the permutation pipeline becomes large, batched, and device-resident enough.



A100 faster region appears by $n=5000$, $p=10,000$, $R=5,000$, $\text{batch_R}=8,192$.

Timing scope

Compile excluded; transfer included; kernel-only excluded.

CPU wins when

p and R are small or random generation, W construction, transfer, collection, or too-small batch_R dominates.

A100 can win when

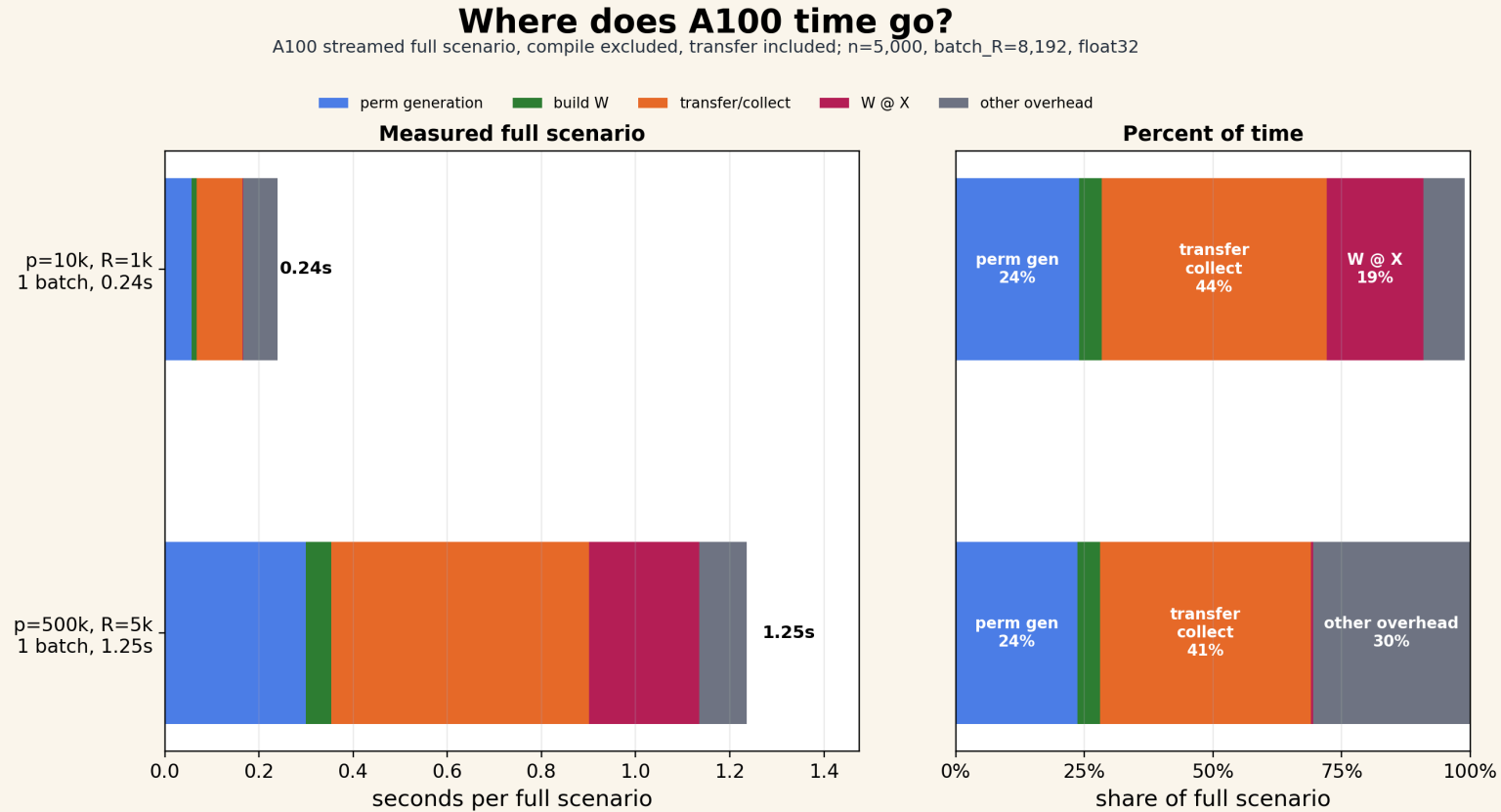
$R \times p$ is large, batch_R fits memory, streamed reduction avoids collecting $R \times p$, and work stays on device.

speedup = matched CPU matrix baseline / A100 streamed full end-to-end.

Shape beats logo: GPU is a formulation decision, not a flag.

Where does A100 time go?

Full-scenario streamed path; compile excluded, transfer included.



Full scenario timing; streamed reduction; batch_R=8,192; residual labeled as other overhead.

A fast kernel is not enough; the full statistical pipeline decides.

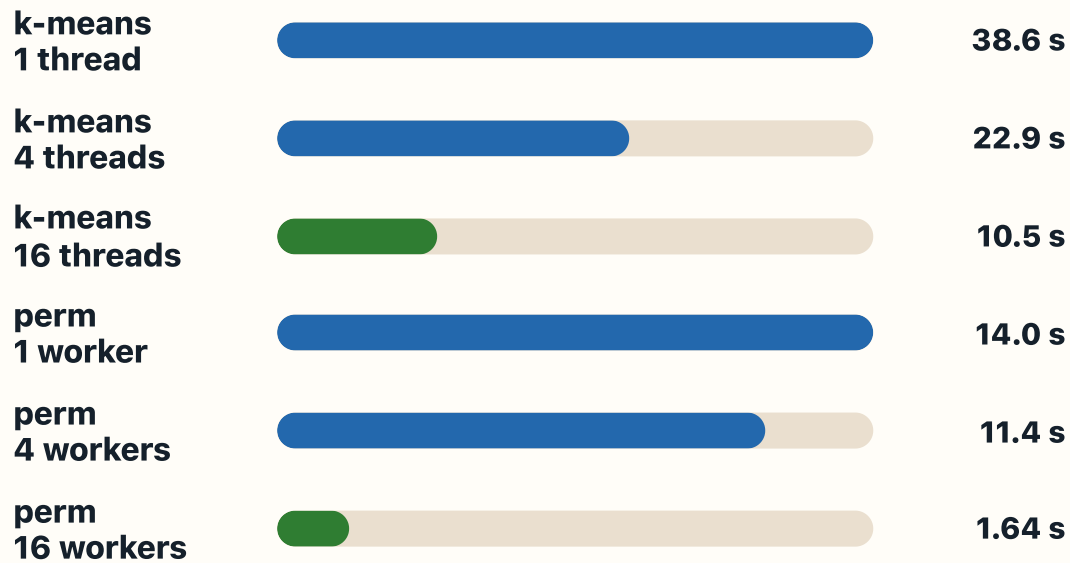
PARALLELISM

CPU parallelism is also a tuning parameter

Linux server CPU sweep, not MacBook.

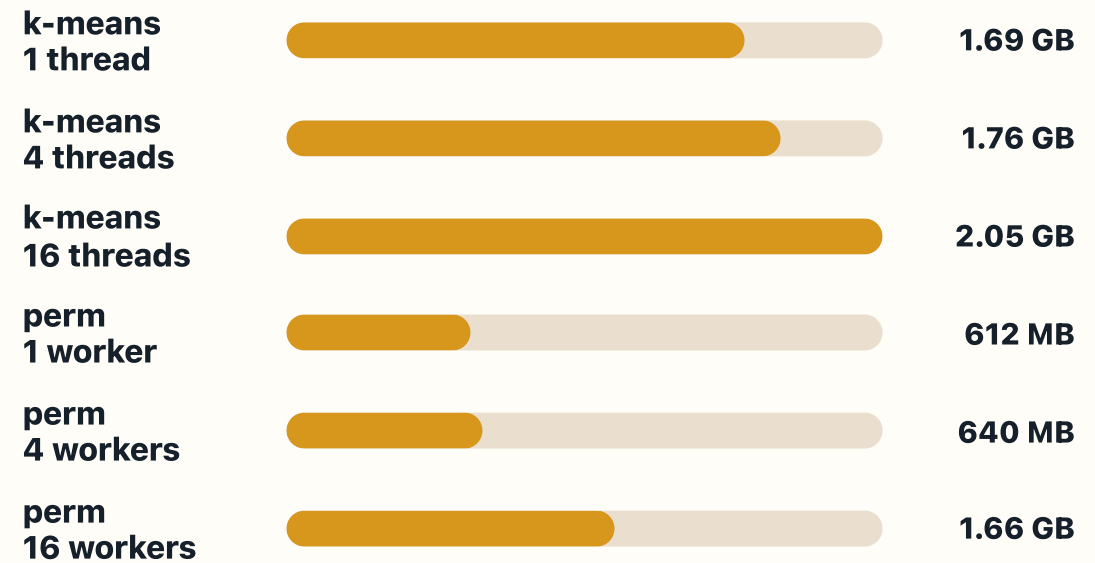
Runtime must be measured

Controlled subset: 1, 4, and 16 threads/workers; lower is better.



Memory keeps a budget attached

Peak host memory from the same sweeps.



More workers is not automatically better. Parallelism has overhead and must be measured.

What each tool did in this talk

Numba

k-means
assignment/update loops

NumPy/BLAS

distance algebra

Threads/Python 3.14

repeated work over shared
arrays

JAX/A100

streamed $W @ X$ after
validation

Choose runtime after simulation identifies the bottleneck.

Codex can generate variants; the statistician owns the claims

Automation layer

- implementation variants
- scenario-grid runners
- environment metadata
- plot regeneration
- README/result manifests

Statistician-owned layer

- statistical target
- null model
- validation criteria
- interpretation of hard cases
- scientific claims

AI can automate experiment hygiene; it cannot define statistical truth.

What did the simulation reveal?

**Start from the validation
result**

**Result not
trustworthy**

keep reference / fix
method

Hot scalar loop

Numba

**Dense matrix
identity**

NumPy/BLAS

**Repeated
shared-array
work**

threads / Python 3.14

**Large batched
device-resident
work**

JAX/A100

Giant temporary

algebraic rewrite

CLOSE

Make the statistic testable. Then make the bottleneck fast.

Clear enough to trust. Fast enough to scale.

`github.com/lucajiang/FastStatisticalModels4Python`

THANK YOU

Thank you



Questions?

Clear enough to trust. Fast enough to scale.

`github.com/LucaJiang/FastStatisticalModels4Python`

Evidence map: each environment answers a different question

MacBook = validation

k-means equivalence, recovery surfaces, permutation p-value equivalence, calibration, and power.

Server CPU = scale and parallelism

k-means shape scaling, Numba thread sweep, permutation R/p scaling, worker and memory sweeps.

A100 = accelerator-shaped pipeline

JAX k-means break-even and streamed W @ X permutation follow-up.

3,840

k-means local pass rows

450

permutation equivalence
pass rows

540

server CPU k-means pass
rows

135

A100 k-means pass rows

Shape stress: K and d become the bottleneck

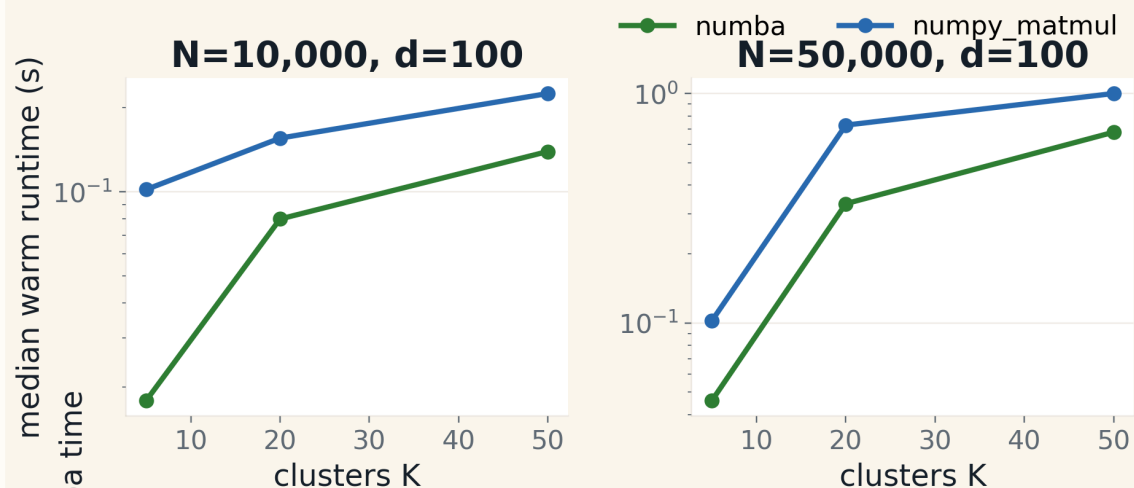
Definitions

N = number of samples

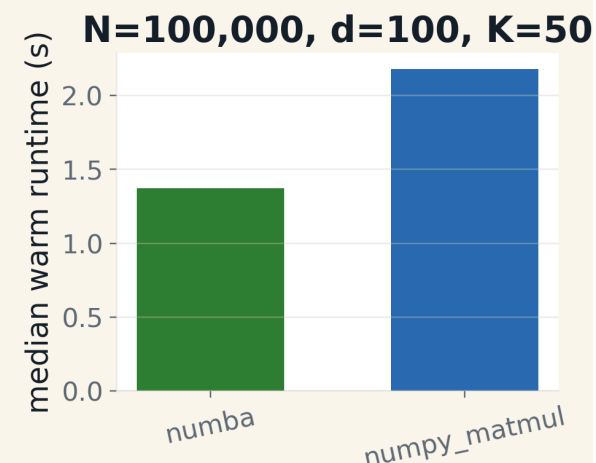
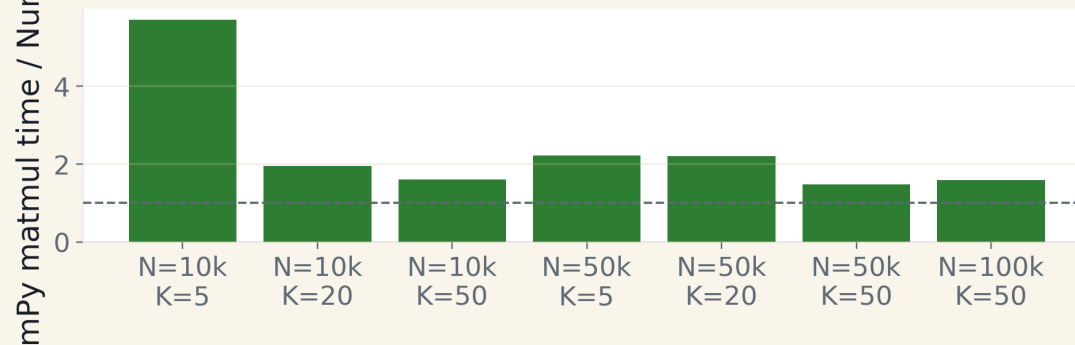
d = number of features /
dimensions

K = number of clusters

k-means shape stress: speed depends on N, K, and d



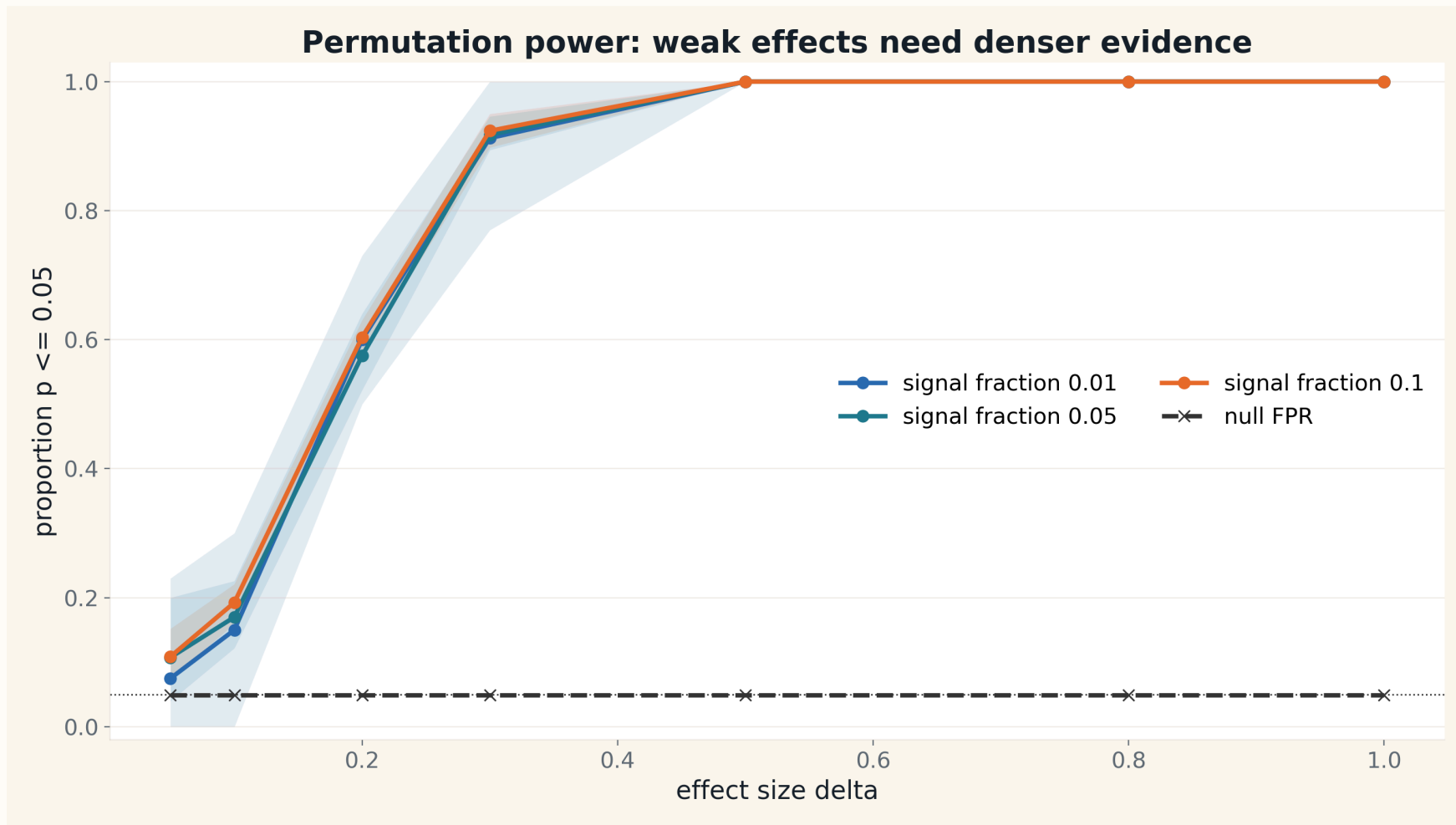
Positive bars mean Numba is faster



1,164 pass rows
max local N 100,000
d=100 stress slice
largest Numba edge 5.7x

Extra MacBook evidence: 1,164 shape-stress rows, all pass.

Power increases with effect size



Backup statistical sanity check: weak effects need denser evidence; larger effects reach high power in this local curve.

What the coding agent automated

Experiment orchestration

Built resumable runners, scenario grids, environment capture, and result manifests.

Visualization surface

Regenerated figures from CSVs and removed charts that looked precise but were unreadable.

Repository surface

Kept reproducible commands, result summaries, and QA notes together.